

Your code is mostly correct and demonstrates a **basic parallelization of BFS and DFS using OpenMP**, but keep in mind: this is *not fully optimal parallel graph traversal*—it uses critical sections heavily, which can limit speedup. Still, it's perfect for learning and viva purposes.

Explanation of the Code

1. Graph Representation

```
class Graph {  
  
    int V;  
  
    vector<vector<int>> adj;
```

- **V** → number of vertices
 - **adj** → adjacency list (vector of vectors)
-

2. Constructor

```
Graph(int V) {  
  
    this->V = V;  
  
    adj.resize(V);  
  
}
```

- Initializes graph with **V** vertices
 - Creates empty adjacency list
-

3. Adding Edges

```
void addEdge(int u, int v) {  
    adj[u].push_back(v);  
    adj[v].push_back(u);  
}
```

- Undirected graph → edge added both ways
-

Parallel BFS

Key Idea:

- Uses **frontier-based BFS**
 - Processes all nodes at current level in parallel
-

Steps:

1. Initialization

```
vector<bool> visited(V, false);
```

```
vector<int> frontier;
```

```
visited[start] = true;
```

```
frontier.push_back(start);
```

- **visited** → track visited nodes
- **frontier** → current level nodes

2. Loop Until Frontier is Empty

```
while (!frontier.empty())
```

3. Print Current Level

```
for (int u : frontier)

    cout << u << " ";
```

4. Parallel Processing

```
#pragma omp parallel
```

Inside this:

(a) Each thread has local storage

```
vector<int> local_next;
```

(b) Parallel loop

```
#pragma omp for nowait schedule(dynamic)
```

- Divides work among threads
- **dynamic** helps load balancing

5. Critical Section (Important)

```
#pragma omp critical
{
    if (!visited[v]) {
        visited[v] = true;
        should_visit = true;
    }
}
```

- Prevents race condition
 - Only one thread updates **visited**
-

6. Merge Results

```
#pragma omp critical
{
    next_frontier.insert(...);
}
```

7. Move to Next Level

```
frontier = next_frontier;
```

Timing

```
double start_time = omp_get_wtime();
```

- Measures execution time
-

Parallel DFS

Key Idea:

- Uses **stack instead of recursion**
 - Parallelizes neighbor exploration
-

Steps:

1. Stack Initialization

```
vector<int> stack;  
  
stack.push_back(start);
```

2. DFS Loop

```
while (!stack.empty())
```

3. Pop Element

```
int u = stack.back();  
  
stack.pop_back();
```

4. Visit Node

```
if (visited[u]) continue;
```

```
visited[u] = true;
```

5. Parallel Neighbor Processing

```
#pragma omp parallel
```

Each thread:

- Checks neighbors
 - Stores in `local_push`
-

6. Merge Neighbors

```
#pragma omp critical
```

```
{  
    to_push.insert(...);  
}
```

7. Push to Stack

```
for (int v : to_push)
```

```
    stack.push_back(v);
```

Important Observations (Very Important for Viva)

- BFS is **level-synchronous**, so easier to parallelize
 - DFS is **inherently sequential**, so parallel version is less efficient
 - `#pragma omp critical` can **reduce performance**
 - This is a **coarse-grained parallel approach**
-

Viva Questions and Answers

◆ Basic Questions

1. What is BFS?

Answer:

Breadth First Search traverses a graph level by level using a queue.

2. What is DFS?

Answer:

Depth First Search explores as far as possible along a branch before backtracking.

3. Difference between BFS and DFS?

BFS	DFS
Uses queue	Uses stack
Level-wise traversal	Depth-wise traversal

Finds shortest path Does not guarantee shortest path

◆ OpenMP Questions

4. What is OpenMP?

Answer:

OpenMP is an API for parallel programming in C/C++ using shared memory.

5. What does `#pragma omp parallel` do?

Answer:

Creates multiple threads to execute code in parallel.

6. What is `#pragma omp for`?

Answer:

Distributes loop iterations among threads.

7. What is `schedule(dynamic)`?

Answer:

Assigns loop iterations dynamically to threads for load balancing.

8. What is a critical section?

Answer:

A block where only one thread executes at a time to avoid race conditions.

◆ Conceptual Questions

9. Why is BFS easier to parallelize than DFS?

Answer:

Because BFS processes nodes level by level, allowing independent parallel work.

10. What is a race condition?

Answer:

When multiple threads access shared data simultaneously causing incorrect results.

11. Why is **visited** inside critical section?

Answer:

To prevent multiple threads from visiting the same node simultaneously.

12. What is a frontier in BFS?

Answer:

The set of nodes at the current level being processed.

13. Why use local vectors (**local_next**, **local_push**)?

Answer:

To reduce contention between threads and improve performance.

14. What is the drawback of this implementation?

Answer:

- Too many critical sections
- Limited scalability
- DFS not truly parallel

15. How can this be optimized?

Answer:

- Use atomic operations instead of critical
 - Use lock-free structures
 - Use parallel graph libraries
-

◆ Output-Based Questions

16. What will be BFS output?

Answer:

0 1 2 3 4 5

17. What will be DFS output?

Answer:

One possible output:

0 2 5 1 4 3

(DFS order can vary)

◆ Advanced Questions

18. What is load balancing?

Answer:

Distributing work evenly across threads.

19. What is scalability?

Answer:

Ability to improve performance with more processors.

20. Can DFS be fully parallelized?

Answer:

No, because it depends on sequential exploration.



Summary

- BFS → Good for parallelization
 - DFS → Limited parallelism
 - OpenMP → simplifies parallel coding
 - Critical sections → necessary but costly
-

If you want, I can also:

- ✓ Simplify this for exam writing
- ✓ Add comments line-by-line in your code
- ✓ Give short 2-mark / 5-mark answers for viva

Just tell me 👍

=====

Your program is actually quite solid—it correctly implements both **sequential and parallel versions** of Bubble Sort and Merge Sort using **OpenMP**, and also measures performance. I'll break it down clearly so you can explain it in a viva.

Code Explanation (Simple + Viva Ready)

1. Header Files

```
#include <iostream>
```

```
#include <vector>
```

```
#include <cstdlib>
```

```
#include <omp.h>
```

```
#include <algorithm>
```

- `iostream` → input/output
 - `vector` → dynamic arrays
 - `cstdlib` → random numbers (`rand()`)
 - `omp.h` → OpenMP for parallel programming
 - `algorithm` → functions like `swap()` and `is_sorted()`
-

2. Utility Functions

Print full array

```
void printArray(const vector<int>& arr)
```

Print first 10 elements

```
void printArray10(const vector<int>& arr)
```

👉 Used to avoid printing huge arrays.

Bubble Sort

Sequential Bubble Sort

```
void sequentialBubbleSort(vector<int>& arr)
```

Logic:

- Compare adjacent elements
- Swap if wrong order
- Repeat for all passes

Time Complexity:

👉 $O(n^2)$

Parallel Bubble Sort (Odd-Even Method)

```
void parallelBubbleSort(vector<int>& arr)
```

Key Idea:

Normal bubble sort **cannot be parallelized directly** due to data dependency.

So we use:

👉 **Odd-Even Transposition Sort**

Two Phases:

1. **Even phase:** (0,1), (2,3), (4,5)
2. **Odd phase:** (1,2), (3,4), (5,6)

```
#pragma omp parallel for
```

👉 Each thread works on independent pairs → no data race

Why it works?

- No two threads access same element simultaneously
 - Safe parallel execution
-

Merge Sort

Merge Function

```
void merge(vector<int>& arr, int left, int mid, int right)
```

Steps:

- Divide into left and right arrays
 - Merge them in sorted order
-

Sequential Merge Sort

```
void sequentialMergeSort(vector<int>& arr, int left, int right)
```

Logic:

1. Divide array
2. Sort left half
3. Sort right half
4. Merge

Time Complexity:

👉 $O(n \log n)$

Parallel Merge Sort (Using OpenMP Tasks)

```
void parallelMergeSortHelper(...)
```

Important Concept: TASKS

```
#pragma omp task
```

👉 Creates lightweight parallel jobs

Why NOT **parallel sections**?

- Would create too many threads
- High overhead
- Can crash system

Threshold Optimization

```
if (right - left < 1000)
```

👉 Small arrays → use sequential sort

👉 Avoid overhead of too many tasks

Synchronization

```
#pragma omp taskwait
```

👉 Wait until both halves are sorted before merging

Main Parallel Function

`#pragma omp parallel`

`#pragma omp single`

👉 Only one thread starts recursion

👉 Others execute tasks

🕒 Performance Measurement

`omp_get_wtime()`

👉 Used to measure execution time

🚀 Speedup Formula

Speedup = Sequential Time / Parallel Time

👉 Shows how much faster parallel version is

🧠 Key Observations

Algorithm	Sequential	Parallel	Speed
	1		

Bubble Sort	Slow	Slightly faster	Limited gain
-------------	------	-----------------	--------------

Merge Sort	Fast	Much faster	Good speedup
------------	------	-------------	--------------

👉 Merge sort benefits more from parallelization

VIVA QUESTIONS & ANSWERS

◆ Basic Questions

Q1: What is OpenMP?

👉 OpenMP is an API for parallel programming in C/C++ using multi-threading.

Q2: What is parallel computing?

👉 Executing multiple tasks simultaneously using multiple processors/cores.

Q3: What is speedup?

👉 Ratio of sequential execution time to parallel execution time.

◆ Bubble Sort Related

Q4: Why is bubble sort hard to parallelize?

👉 Because of data dependency between adjacent elements.

Q5: What technique did you use?

👉 Odd-Even Transposition Sort.

Q6: Time complexity of bubble sort?

👉 $O(n^2)$

◆ Merge Sort Related

Q7: Why is merge sort suitable for parallelization?

👉 Because it uses divide-and-conquer, allowing independent subproblems.

Q8: What are OpenMP tasks?

👉 Lightweight threads used for recursive parallelism.

Q9: Why use `taskwait`?

👉 To ensure both halves are sorted before merging.

Q10: Time complexity of merge sort?

👉 $O(n \log n)$

◆ Performance

Q11: Why parallel merge sort is faster than parallel bubble sort?

👉 Merge sort divides work efficiently, while bubble sort has many dependencies.

Q12: What is overhead in parallel computing?

👉 Extra time required for thread creation and management.

Q13: Why use threshold (1000)?

👉 To avoid creating too many small tasks.

◆ Advanced

Q14: What is a data race?

👉 When multiple threads access and modify the same data simultaneously.

Q15: What is `#pragma omp parallel for`?

👉 Splits loop iterations across multiple threads.

Q16: Difference between `parallel for` and `task`?

parallel for

task

Loop-based

Recursive/independent work

Static division

Dynamic scheduling



Tips for Viva

- Say "**odd-even transposition**" confidently → examiner impressed
 - Mention "**task-based parallelism**"
 - Highlight "**threshold optimization**"
 - Emphasize "**data race avoidance**"
-

If you want, I can also:

- ✓ Draw diagrams for both algorithms
- ✓ Give expected output
- ✓ Help you write theory for your journal/report



Why Bubble Sort Speedup is Low

Even after parallelization:

- Still $O(n^2)$
- Many passes required
- Synchronization after each phase



So improvement is **limited**



Why Merge Sort Speedup is High

- Work is divided recursively
- Independent subproblems
- Less synchronization



Ideal for parallel systems



Task Scheduling (Important)

#pragma omp task



Tasks are:

- Not executed immediately

- Put in a queue
- Picked by available threads

Viva Line:

👉 “OpenMP uses a work-stealing scheduler for task execution.”



Memory Usage Insight

Merge sort:

- Uses extra arrays `L[]` and `R[]`
- Space complexity: $O(n)$

Bubble sort:

- In-place sorting
 - Space complexity: $O(1)$
-



Load Balancing

👉 Merge sort automatically balances load:

- Each thread gets similar sized subarrays

👉 Bubble sort:

- Work not evenly distributed
-



TRICKY VIVA QUESTIONS (High Probability)

◆ Conceptual Deep Questions

Q1: What happens if we remove `#pragma omp taskwait`?

👉 Answer:

- Merge may execute before sorting completes
 - Leads to incorrect output
-

Q2: Why not parallelize merge function?

👉 Answer:

- Merge step is inherently sequential
 - Parallel merge is complex and adds overhead
-

Q3: What is false sharing?

👉 Answer:

- When threads modify variables in same cache line
 - Causes performance degradation
-

Q4: What is granularity?

👉 Answer:

- Size of tasks in parallel computing

👉 Your code:

- Uses **coarse-grained parallelism** (big tasks)
-

Q5: What if threshold is too small?

👉 Answer:

- Too many tasks created
 - Overhead increases
 - Performance decreases
-

Q6: What if threshold is too large?

👉 Answer:

- Less parallelism
 - Underutilization of CPU
-

◆ Code-Based Questions

Q7: Why `#pragma omp single` is used?

👉 Answer:

- Ensures only one thread starts recursion
 - Prevents duplicate task creation
-

Q8: Why not use `parallel for` in merge sort?

👉 Answer:

- Merge sort is recursive
 - Not loop-based → tasks are better
-

Q9: What does `shared(arr)` mean?

👉 Answer:

- All threads access same array
 - Needed for correct updates
-

Q10: What is default variable scope in OpenMP?

👉 Answer:

- Global variables → shared
- Local variables → private

◆ Analytical Questions

Q11: Compare both algorithms

Feature	Bubble Sort	Merge Sort
Complexity	$O(n^2)$	$O(n \log n)$
Parallel Efficiency	Low	High
Memory	$O(1)$	$O(n)$
Practical Use	Rare	Widely used

Q12: Which algorithm is better for large data?

👉 Merge Sort (especially parallel version)

Q13: Why do we check `is_sorted()`?

👉 Answer:

- To verify correctness of parallel execution
-



Experimental Questions (VERY IMPORTANT)

Q14: How did you measure performance?

👉 Using:

`omp_get_wtime()`

Q15: What factors affect performance?

👉 Answer:

- Number of threads
 - Input size
 - Hardware (cores)
 - Task overhead
 - Memory access
-

Q16: What is ideal speedup?

👉 Answer:

- Ideally = number of cores
 - Practically less due to overhead
-



SUPER ADVANCED (Impress Examiner)

Q17: What is Amdahl's Law?

👉 Answer:

- Limits speedup of parallel programs

Formula:

$$\text{Speedup} = 1 / (S + (1-S)/P)$$

Where:

- S = sequential portion
 - P = processors
-

Q18: What is scalability?

👉 Answer:

- Ability to improve performance with more processors

👉 Merge sort → scalable

👉 Bubble sort → not scalable

Q19: Difference between concurrency and parallelism?

👉 Answer:

- Concurrency → multiple tasks in progress
 - Parallelism → tasks executed simultaneously
-

Q20: What is thread overhead?

👉 Answer:

- Time spent creating and managing threads
-

COMMON MISTAKES (You Can Mention)

- Using parallel sections in recursion ❌
- No threshold ❌

- Forgetting `taskwait` ❌
 - Too many threads ❌
-



SHORT THEORY (For Journal)

👉 You can write this:

Parallel Bubble Sort is implemented using odd-even transposition to avoid data races. It alternates between even and odd indexed comparisons, allowing safe parallel execution.

Parallel Merge Sort uses OpenMP tasks to divide the problem recursively. Tasks are scheduled dynamically among threads, improving performance. A threshold is used to switch to sequential execution for small subarrays to reduce overhead.



FINAL VIVA TIP

If examiner asks:

👉 “Which one would you use in real life?”

Answer confidently:

👉 “Parallel Merge Sort, because it has better time complexity and scales efficiently with multiple cores.”

=====

This program demonstrates **Parallel Reduction using OpenMP** to compute **Minimum, Maximum, Sum, and Average** of a large array, and compares **sequential vs parallel performance**.



Code Explanation

1. Header Files

```
#include <iostream>
```

```
#include <vector>
```

```
#include <omp.h>
```

```
#include <cstdlib>
```

- `iostream` → for input/output
 - `vector` → dynamic array
 - `omp.h` → OpenMP library for parallel programming
 - `cstdlib` → for `rand()`
-

2. Initialize Data

```
int n = 1000000;
```

```
vector<int> nums(n);
```

```
for(int i=0;i<n;i++){
```

```
    nums[i] = rand() % 10000;
```

```
}
```

- Creates an array of **1 million elements**
 - Each element is a random number between **0–9999**
-

3. Sequential Computation

```
mins, maxs = nums[0];
```

```
sums = 0;
```

⚠ Small mistake:

- This does NOT assign both values properly
- Correct form:

```
mins = maxs = nums[0];
```

Loop:

```
for(int i=0;i<n;i++){  
  
    if(nums[i]<mins)  
  
        mins = nums[i];  
  
    if(nums[i]>maxs)  
  
        maxs = nums[i];  
  
    sums+=nums[i];  
  
}
```

- Finds:
 - Minimum value
 - Maximum value
 - Sum of all elements

Average:

```
avgs = (double) sums/n;
```

Time Measurement:

```
start = omp_get_wtime();
```

```
end = omp_get_wtime();
```

- Measures execution time
-

4. Parallel Computation (OpenMP Reduction)

```
#pragma omp parallel for reduction(min: minp) reduction(max: maxp) reduction(+:sump)
```

What happens here?

- Loop is divided among multiple threads
- Each thread computes:
 - Local min
 - Local max
 - Local sum

Then OpenMP **combines results using reduction**.

Parallel Loop:

```
for(int i=0;i<n;i++){  
    if(nums[i]<minp)  
        minp = nums[i];  
    if(nums[i]>maxp)  
        maxp = nums[i];  
    sump+=nums[i];  
}
```

Average:

```
avgp = (double) sump/n;
```

5. Output

- Prints:
 - Sequential results
 - Parallel results
 - Execution time
 - Speedup

Speedup = Sequential Time / Parallel Time

Key Concept: Reduction

Reduction = combining results from multiple threads safely

Types used:

- `reduction(min: minp)` → finds global minimum
 - `reduction(max: maxp)` → finds global maximum
 - `reduction(+: sump)` → sums values
-

Important Corrections

Fix initialization:

```
mins = maxs = nums[0];
```

```
minp = maxp = nums[0];
```

Viva Questions & Answers

Basic Questions

1. What is parallel reduction?

Answer:

It is a technique where multiple threads compute partial results and combine them into a final result.

2. What is OpenMP?

Answer:

OpenMP is an API for parallel programming in C/C++ using compiler directives.

3. What does `#pragma omp parallel for` do?

Answer:

It splits loop iterations across multiple threads for parallel execution.

4. Why is reduction needed?

Answer:

To avoid **race conditions** when multiple threads update the same variable.

5. What is a race condition?

Answer:

When multiple threads access and modify shared data simultaneously causing incorrect results.

Intermediate Questions

6. What does `reduction(+: sump)` mean?

Answer:

Each thread keeps a local copy of `sump` and adds them at the end.

7. Why use `omp_get_wtime()`?

Answer:

To measure execution time in parallel programs.

8. What is speedup?

Answer:

$\text{Speedup} = \text{Time}(\text{sequential}) / \text{Time}(\text{parallel})$

9. Why is parallel faster?

Answer:

Because multiple threads execute simultaneously using multiple CPU cores.

10. Can parallel ever be slower?

Answer:

Yes, due to:

- Thread overhead
 - Small data size
 - Synchronization cost
-

Advanced Questions

11. What happens without reduction?

Answer:

Incorrect results due to race conditions.

12. How many threads are used?

Answer:

Depends on system (usually number of CPU cores).

13. How to set number of threads?

```
omp_set_num_threads(4);
```

14. Difference between sequential and parallel?

Sequential	Parallel
Single thread	Multiple threads
Slower	Faster
No synchronization issues	Needs synchronization

15. What is scalability?

Answer:

Ability of program to improve performance with more processors.



Final Summary

- Sequential → simple but slower
- Parallel (OpenMP) → faster using multiple cores
- Reduction → ensures correct results in parallel
- Speedup shows performance improvement

=====

I'll walk you through this **step by step**, from the setup commands to the CUDA program, explaining each line in simple terms.

◆ Part 1: Setup Commands (Jupyter/Colab)

✓ `!nvcc --version`

- `!` → runs a **shell command** inside notebook
- `nvcc` → NVIDIA CUDA compiler
- `--version` → checks if CUDA is installed

👉 Output shows CUDA version.

✓ `!pip install nvcc4jupyter`

- Installs a tool to run CUDA code inside Jupyter notebook

✓ `%load_ext nvcc4jupyter`

- Loads the extension so you can run CUDA code using special commands

✓ `%%writefile cuda_program.cu`

- Saves the code into a file named `cuda_program.cu`

- `.cu` → CUDA source file
-

◆ Part 2: Header and Setup

```
#include <iostream>
#include <cuda.h>
```

- `<iostream>` → for input/output (`cout`, `cin`)
 - `<cuda.h>` → CUDA library
-

```
using namespace std;
```

- Allows writing `cout` instead of `std::cout`
-

```
#define BLOCK_SIZE 2
```

- Defines a constant: each block has **2 threads** (or **2x2 later**)
-

◆ Part 3: Vector Addition Kernel (GPU Function)

```
__global__ void vectorAdd(int *A, int *B, int *C, int N)
```

- `__global__` → runs on **GPU**, called from CPU
 - `A`, `B` → input arrays
 - `C` → result array
 - `N` → size
-

```
int i = blockIdx.x * blockDim.x + threadIdx.x;
```

- Calculates **global index** of thread
- `blockIdx.x` → block number
- `blockDim.x` → threads per block
- `threadIdx.x` → thread index inside block

👉 Formula gives unique position for each thread

```
if (i < N)
    C[i] = A[i] + B[i];
```

- Prevents **out-of-bounds access**
 - Each thread computes:
👉 `C[i] = A[i] + B[i]`
-

◆ Part 4: Matrix Multiplication Kernel

```
__global__ void matrixMul(float *A, float *B, float *C, int N)
```

- GPU function for matrix multiplication
-

```
int row = blockIdx.y * blockDim.y + threadIdx.y;
int col = blockIdx.x * blockDim.x + threadIdx.x;
```

- Computes **row and column index**
 - Each thread handles **one element of result matrix**
-

```
float sum = 0.0f;
```

- Stores result for one cell
-

```
for (int n = 0; n < N; ++n)
    sum += A[row * N + n] * B[n * N + col];
```

- Performs dot product:
 - Row of A $\times$ Column of B
-

```
C[row * N + col] = sum;
```

- Stores result in matrix C
-

◆ Part 5: Vector Addition Function (CPU Side)

```
void runVectorAddition()
```

- Function to execute vector addition
-

```
int N;  
cin >> N;
```

- Takes input size
-

```
int size = N * sizeof(int);
```

- Total memory needed
-

Host Memory Allocation

```
int *hA = new int[N];
```

- Allocates memory on CPU
-

```
hA[i] = i;  
hB[i] = i * 2;
```

- Initializes arrays
-

Device Memory Allocation

```
cudaMalloc(&dA, size);
```

- Allocates memory on GPU
-

```
cudaMemcpy(dA, hA, size, cudaMemcpyHostToDevice);
```

- Copies data from CPU → GPU
-

Kernel Launch

```
int numBlocks = (N + BLOCK_SIZE - 1) / BLOCK_SIZE;
```

- Ensures enough blocks to cover all elements
-

```
vectorAdd<<<numBlocks, BLOCK_SIZE>>>(dA, dB, dC, N);
```

- Launches kernel:
 - `numBlocks` → number of blocks
 - `BLOCK_SIZE` → threads per block
-

```
cudaMemcpy(hC, dC, size, cudaMemcpyDeviceToHost);
```

- Copies result back GPU → CPU
-

```
delete[] hA;  
cudaFree(dA);
```

- Frees memory (CPU + GPU)
-

◆ Part 6: Matrix Multiplication Function

```
int K, N;  
N = K * BLOCK_SIZE;
```

- Matrix size depends on K
-

```
float *hA = new float[N * N];
```

- Allocates matrix
-

```
hA[j * N + i] = 2;
```

- Initializes matrix A with 2
-

```
dim3 threadBlock(BLOCK_SIZE, BLOCK_SIZE);
```

- Defines block size (2×2 threads)
-

```
dim3 grid(K, K);
```

- Defines grid size
-

```
matrixMul<<<grid, threadBlock>>>(dA, dB, dC, N);
```

- Launch kernel
-

◆ Part 7: Main Function

```
int main()
```

- Entry point

```
runVectorAddition();  
runMatrixMultiplication();
```

- Calls both functions

```
return 0;
```

- Ends program
-

◆ Part 8: Compile & Run

```
!nvcc cuda_program.cu -o cuda_program && ./cuda_program
```

- `nvcc` → compiles CUDA code
 - `-o cuda_program` → output executable
 - `./cuda_program` → runs program
-

◆ Final Understanding

◆ Vector Addition

- Each thread adds one element

◆ Matrix Multiplication

- Each thread computes one cell

Key CUDA Concepts Used

- Kernel (`__global__`)
- Threads, Blocks, Grid
- Memory transfer (Host ↔ Device)
- Parallel computation

If you want, I can:

- ✓ Draw diagram of thread/block working
- ✓ Convert this into viva answers
- ✓ Simplify into short notes for exam

1. What is CUDA?

Answer: CUDA is a parallel computing platform by NVIDIA that allows execution of programs on GPU for faster computation.

2. What is nvcc?

Answer: `nvcc` is the CUDA compiler used to compile `.cu` files.

3. Difference between CPU and GPU?

Answer:

- CPU → Sequential processing, few cores
- GPU → Parallel processing, many cores

4. What is a kernel?

Answer: A kernel is a function that runs on the GPU and executes in parallel.

5. What does `__global__` mean?

Answer: It indicates a function runs on GPU and is called from CPU.

6. Explain kernel launch syntax:

```
vectorAdd<<<numBlocks, BLOCK_SIZE>>>>(dA, dB, dC, N);
```

Answer:

- `numBlocks` → number of blocks
 - `BLOCK_SIZE` → threads per block
-

7. What is `threadIdx`?

Answer: Index of a thread inside a block.

8. What is `blockIdx`?

Answer: Index of a block inside the grid.

9. What is `blockDim`?

Answer: Number of threads per block.

10. How is global index calculated?

Answer:

```
i = blockIdx.x * blockDim.x + threadIdx.x
```

◆ Vector Addition

11. How does vector addition work in GPU?

Answer: Each thread computes one element:

```
C[i] = A[i] + B[i]
```

12. Why use `if (i < N)`?

Answer: To avoid accessing memory beyond array size.

13. Why use `(N + BLOCK_SIZE - 1) / BLOCK_SIZE`?

Answer: To round up and ensure all elements are covered.

◆ Matrix Multiplication

14. How is matrix multiplication parallelized?

Answer: Each thread computes one element of result matrix.

15. How are row and column calculated?

Answer:

- `row = blockIdx.y * blockDim.y + threadIdx.y`
 - `col = blockIdx.x * blockDim.x + threadIdx.x`
-

16. What does the loop do in matrix multiplication?

Answer: Computes dot product of row of A and column of B.

◆ Memory Management

17. What is host memory?

Answer: Memory on CPU.

18. What is device memory?

Answer: Memory on GPU.

19. What does `cudaMalloc` do?

Answer: Allocates memory on GPU.

20. What does `cudaMemcpy` do?

Answer: Copies data between CPU and GPU.

21. Types of `cudaMemcpy`?

Answer:

- Host → Device
 - Device → Host
 - Device → Device
-

◆ Execution & Performance

22. Why is GPU faster?

Answer: Because many threads execute tasks in parallel.

23. What is block size?

Answer: Number of threads in one block.

24. What happens if block size is too large?

Answer: It may exceed GPU limits or reduce performance.

◆ Advanced

25. What is `dim3`?

Answer: A datatype used to define 3D grid/block dimensions.

26. What is thread hierarchy?

Answer: Threads → Blocks → Grid

27. Limitations of this code?

Answer:

- No shared memory
 - Not optimized
 - No error handling
-

28. How to optimize this code?

Answer:

- Use shared memory
- Use tiling

- Increase block size
-

◆ Output-Based

29. Why is output 48 in matrix multiplication?

Answer:

Each element = sum of (2×4) repeated N times $\rightarrow 48$

◆ Tricky Questions

30. Can GPU run without CPU?

Answer: No.

31. What if `cudaMemcpy` is not used?

Answer: Data won't transfer between CPU and GPU.

32. What is synchronization?

Answer: Ensuring all threads complete before moving ahead.

Exam Tip

If you forget an answer, say:

👉 "Each thread processes one element, enabling parallel execution."